From Platform to Product

A self-directed AI animation case study

Julian Visser — Project Manager & Writer
visserjulian@gmail.com · Portfolio · Source code · 2026

Origin

Since 2024, I've been watching AI image and video generation developed extremely fast, from a technological standpoint, that it seemed initially obvious where it's heading: replication of professional TV/Film standards. But I was aware that current AI output in 2025 and 2026 isn't at all up to professional TV-Film production standards. At least, AI has yet to replicate the highest-quality standards that I have ever seen in media: Netflix's Arcane, Dreamwork's Wild Robot. My hypothesis was that AI video-image creation will eventually reach this level of replication and technical maturity sooner than anticipated and it's going to slash the cost of producing animated content. Therefore, the people who have learned how to best use this new AI technology will have an edge over the studios that started learning later. That is the basis of why I started this case study: to see if I too can create an animation pipeline.

To be clear about what "I built" means here: I'm a writer, not a trained coder, and I don't have a CS degree. I don't write production code from scratch — AI wrote the majority of the code you'll see referenced in this case study, and my job was to specify what to build, review what came back, and correct it rather than to type it all.

The ambitious version

My first attempt in this case study was a fully modular, provider-agnostic pipeline. This pipeline was to contain all production phases: pre-production, production, and post-production. All controllable, all swappable with different AI APIs whenever possible. If a better AI image model ships tomorrow, my model would simply switch to the API of that new and improved AI model. This switch included all settings on deterministic outputs, contract validation, atomic writes, the works. I spent months on it. 1,711 tests pass in the full private build — 1,695 in the public snapshot you can inspect on this portfolio.

The problem I didn't see coming

Movie, TV, and video studios don't want a general modular pipeline that can fit to any kind of studio workflow. They have their own connections, their own specific software, their own specialised workflows. A general-purpose pipeline tool doesn't slot cleanly into a specific studio's specific workflows. It's interesting on paper but hard to sell in a room. On top of that: my project had no warm agency introductions, no committed budget for the training data which is key to any AI models and pipelines. Anyone can train a style model. All in all, I was building a platform without a buyer. I was chasing an exciting new tech development that sat on the horizon without considering that I might run off a cliff.

The narrower version

So my pivot would be to refocus all the work done down to a narrower scope instead of starting over from scratch. New scope: prose script in, short animated storyboard out. Instead of focusing on all phases of productions, I would pivot to a pre-production specialisation. Ideally a sixty to ninety seconds video transcript. A specific, demo-able thing you can attach to a cold email. The underlying code from the original build got reorganized into a shared module and reused as the foundation.

The re-scope is meant to reorganize the original build into a working scaffold: same engineering, narrower target. Architecture and test coverage are in place. What I salvaged can confirm what the modular architecture was supposed to allow — cutting down scope without starting over.

A few things I've held onto through this case study

Don't take shortcuts. AI is very good at chasing whatever goal you set, and less good at checking whether the goal makes sense downstream. If you don't systemize the verification between stages, you get plausible-looking output that breaks under close reading. I learned this the hard way once: I ended up with two parallel builds of the same pipeline running at the same time because I wasn't paying enough attention to what was actually being changed. Cleaning that up cost time that I couldn't get back.

Show your work. This is a self-directed project. I don't have academic degrees or professional credentials to point at in relation to coding or calibrating AI tools. What I can point at is the process. The plans, the failed attempts, the specific moments where I course-corrected, the decisions made under uncertainty located in my GitHub. Blue-collar workers can show callouses. My work leaves less visible evidence, so I have to be more deliberate about publishing it.

AI is an assistant, not a replacement. AI is genuinely good at software iteration, troubleshooting, and finding edge cases you didn't think of before. It's much worse at anything

requiring self-expression or original insight. The industry framing that AI "writes" or "creates" misses what AI is actually doing. It copies. It recombines what already exists in its training data. That's useful. It's just not the same thing as creating, feeling and thinking. If you don't learn as much of the distinctions and nuances in AI, then you are liable to misuse AI in ways that it is not designed for. AI is a tool, or a really smart intern that knows everything but can't be trusted to act independently in ways that align with what you want.

Calibrate the tool before you use it. The single most productive change I made during this case study was insisting the AI tell me when it was uncertain, when it calculated whenever my premise was wrong, and what alternatives it saw. No gas-lighting, no echo-chambering, no blind agreement and no encouragement. Confidence labels, per-claim assumptions, skeptical review of my own reasoning. That reshaped what I got out of the tooling more than any prompt trick did.

Discipline over shortcuts. There aren't any. Using AI and working with AI is a different kind of hard work, not less of it. Anyone that tells you AI makes work easier, is lying to you. AI is something that you learn and know how to use, just like any other device. Otherwise, you are setting yourself up to fail. Efficiency and speed don't automatically give consistent desired quality. What I actually want is to make quality work, and quality still costs what it costs in time, learning and trial-and-error.

Why I stopped here

The case study is the deliverable. Continuing this specific product any further would need real customer discovery paired with an animation professional who has the studio contacts that I currently don't have as of time of writing this paper. That's a partnered venture, not more solo engineering. I'm publishing this to close the artifact, not as a promise to keep iterating on it.

Full technical case study attached (PDF). The code itself is public: https://github.com/Vhaser-system/piper-animatic — a frozen portfolio snapshot that installs standalone and passes its own test suite.